

i Forside**UNIVERSITETET I OSLO****Det matematisk-naturvitenskapelige fakultet****Written exam in IN1900****2024 Fall****Date and time: Monday 16/12 from 09:00 to 13.00 (4 hours)****Permitted aides: None****A calculator is available in Inspera****It is important that you read this information carefully before you start answering the questions.****If you need to zoom in the exam set, hold ctrl and push + eller - on the numeric keyboard.**

The exam contains multiple choice questions and text questions where you shall write short programs or read programs and write the output from the program. There are 14 questions that should be answered, and in total 75 points available on these questions. Question 15 shall not be answered, but is used by the examiners during the grading of the exam, to include the points from the mid term exams.

The number of points available is specified for each question. For questions with multiple sub-questions, each sub-question has the same number of points. On multiple choice questions you get the same score (0) for a wrong answer and for not answering at all, so you should always mark an answer.

If you are missing information you can make your own reasonable assumptions, as long as they are in line with the "nature" of the question. In text questions you should then specify the assumptions you made, for instance in comments to the code.

All code in the question texts is written in Python 3.

Most of the questions lead to short code with little need for comments, unless you do something complicated or non-standard (which is not recommended; but in this case the comments should explain the ideas behind the program to make it easier to evaluate the code).

A question may ask you to write a function. A main program which calls the function is in this case not needed, unless it is specifically asked for in the question text.

1 Hva skrives ut?

What is printed when the following code is run?

```
def absolute(x):  
    if x < 0:  
        return -x  
    return x
```

```
for x_ in [-3,4,0]:  
    print(absolute(x_), end=' ')
```

Select one alternative:

- ☐ 3 -4 0
- ☐ An error message
- ☐ 3 -3 4 0
- ☒ 3 4 0



Maximum marks: 2

2 Lister

If *a* is an integer ('int'), which of the following lines will *not* result in a list of length *a*:

Select one alternative:

- ☒ `b = [3*e for e in range(a+1)]`
- ☐ `b = [3*e for e in range(1,a+1)]`
- ☐ `b = list(range(a))`
- ☐ `b = [0]*a`
- ☐ `b = list(range(a-1)) + [a]`



Maximum marks: 2

3 Lesing av fil

A file "constants.txt" has the following contents (there are no blank lines in the file):

name of constant	value	unit
light speed	299792458.0	m/s
gravitational constant	6.67259e-11	m**3/kg/s**2
Planck constant	6.6260755e-34	J*s
elementary charge	1.60217733e-19	C
Avogadro number	6.0221367e23	1/mol
Boltzmann constant	1.380658e-23	J/K
electron mass	9.1093897e-31	kg
proton mass	1.6726231e-27	kg

When the program below is run, an error message is printed and the program stops. In which line does the program stop?

```
constants = {}
with open("constants.txt") as infile:
    for line in infile:
        words = line.split()
        value = float(words[2])
        unit = words[3]
        name = " ".join(words[:2])
        constants[name] = (value, unit)
```

Select one alternative:

- ☒ value = float(words[2])
- ☐ name = " ".join(words[:2])
- ☐ constants[name] = (value, unit)
- ☐ unit = words[3]



Maximum marks: 2

4 Feilhåndtering (exceptions)

The program **check.py** looks as follows:

```
import sys
try:
    # One line is missing here
except IndexError:
    print("Feil A")
    sys.exit()
except (ValueError, TypeError):
    print("Feil B")
    sys.exit()
except ZeroDivisionError:
    print("Feil C")
    sys.exit()
```

The program is missing one line. We replace the missing line with one of the alternatives on the left side of the table below, and run the program with the command

Terminal> python check.py 4 0

Decide for each row in the table what is printed by the program. "No output" means that nothing is printed.

Please match the values:

	Feil A	Feil B	Feil C	No output
<code>j = sys.argv[1]</code>	☐	☐	☐	☒
<code>j,k = sys.argv[2], sys.argv[3]</code>	☒	☐	☐	☐
<code>j = float(sys.argv[0])</code>	☐	☒	☐	☐
<code>h = 1/int(sys.argv[2])</code>	☐	☐	☒	☐
<code>k = sys.argv[:3]</code>	☐	☐	☐	☒
<code>h = sys.argv[2] / sys.argv[1]</code>	☐	☒	☐	☐

Maximum marks: 3

5 Hva skrives ut?

What is printed when the following code is run?

```
class Y:
    def __init__(self, v0):
        self.v0 = v0
        self.g = 9.81

    def __call__(self, t):
        return self.v0*t - 0.5 * self.g * t**2

    def __str__(self):
        return f"Ball formula, v0 = {self.v0}"

y = Y(5)
print(y)
```

Fill in your answer here

Format

B

I

U

x_2

x^2

I_x

Ball formula, v0 = 5

Words: 0

Maximum marks: 2

6 Beregning av en sum

a) The following sum is an approximation of the exponential function:

$$f(x) = \sum_{k=0}^n \frac{x^k}{k!}$$

Write a Python function `exp_approx(x,n)` that implements this formula and returns the computed value. The argument `x` can be a floating point number (scalar) or an array of floats, and `n` is a positive integer. Recall that $k!$ is the factorial of k . Include necessary imports.

b) Write a test function for the function in a). You can choose `n=10` and `x=1`, and assume that the difference between the approximation and the exact exponential function is then less than 10^{-4} . You can use the `exp` function from `math` or `Numpy` to compute the exact value.

Fill in your answer here

1	Solution at the end of the exam set.
---	--------------------------------------

Maximum marks: 5

7 Lister og arrays

We want to create lists and/or arrays containing function values for the function

$$f(x) = \ln(x)$$

We want two lists/arrays; x and y , which contain the x -values and the corresponding function values $y = f(x)$. Both x and y shall have length 101, and the x -values shall be uniformly distributed (equal distance) in the interval $[1,10]$.

- a) Write code that uses a for-loop to fill x and y with the correct values. Here x and y can be either lists or arrays.
- b) Do the same operation as in a), but now without using a loop. Both x and y must now be Numpy arrays.

Include necessary imports on both sub-questions.

Fill in your answer here

1	Solution at the end of the exam set.
---	--------------------------------------

Maximum marks: 5

8 Arealet av en trekant

The area of a triangle is given by the formula

$$A = \frac{1}{2} |x_2 y_3 - x_3 y_2 - x_1 y_3 + x_3 y_1 + x_1 y_2 - x_2 y_1|$$

where (x_1, y_1) , (x_2, y_2) , (x_3, y_3) are the coordinates of the three corners of the triangle. The following test function is meant to test a function **triangle_area(vertices)**, which uses the formula above to compute the area of an arbitrary triangle.

```
def test_triangle_area():
    v1 = [0,0]; v2 = [1,0]; v3 = [0,2]
    vertices = [v1, v2, v3]
    expected = 1
    computed = triangle_area(vertices)
    tol = 1e-10
    success = abs(expected - computed) < tol
    assert success
```

Write the function **triangle_area(vertices)** so that the function works as intended and the test passes.

Fill in your answer here

1

Solution at the end of the exam set.

Maximum marks: 4

9 Numerisk derivert

The derivative of a function $f(x)$ can be estimated with the formula

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

a) Write a class **Diff** with two methods:

- A constructor (`__init__`), which takes the following arguments: a Python-function f , which is the function we want to estimate the derivative of, and an argument h which is the step length in the formula above. The argument h can be made optional, with a default value of $h=1e-4$.
- A `__call__`-method that takes an argument x , and uses the formula above to estimate and return the derivative of the function in x .

b) Write code that uses the class from a) to estimate and plot the derivative of $\sin(2\pi x)$ for x in the interval $[0,1]$. Include necessary imports.

Fill in your answer here

1

Solution at the end of the exam set.

Maximum marks: 8

10 Stykkvis konstant funksjon

A piecewise constant function can be defined as follows:

$$f(x) = \begin{cases} a_0 & \text{for } x < b_0, \\ a_n & \text{for } b_{n-1} \leq x < b_n, n = 1, \dots, N-1, \\ a_N & \text{for } x \geq b_{N-1}. \end{cases}$$

where a og b er lists of numbers. The list a has length $N+1$ and b has length N , and we have that $b_{n+1} > b_n$ for all n .

a) Write a class **Piecewise** which represents such a function. The class shall have a constructor which takes the two lists a and b as arguments, and a `__call__`-method which takes x as argument and returns the value $f(x)$. You can assume that the argument x is a number (not a Numpy array).

b) Write a test function which tests the class from a). Use the lists $b = [10, 20]$ and $a = [1, 2, 3]$ as arguments, og compare the results from the class with the expected values for $x = 5$ and $x = 15$. (Remember that you can write the test function even you have not completed sub-question a), since you know what the class is supposed to do.)

Fill in your answer here

1	Solution at the end of the exam set.
---	--------------------------------------

Maximum marks: 10

11 Klasse for vektorer i planet

We want to implement a class **Vec2D**, which represents vectors in the plane. Usage of the class is illustrated in the following interactive Python session:

```
>>> x = Vec2D([1,2])
>>> y = Vec2D([4,4])
>>> z = x + y
>>> print(z)
(5,6)
>>> print(type(z))
<class '__main__.Vec2D'>
>>> print(x * y)
12
```

The operation $x * y$ here returns the inner product of the vectors x and y , that is $x_1y_1 + x_2y_2$. That is, addition of two vectors shall result in a new **Vec2D**-instance, while multiplication of two vectors yields a number.

Write the code for the class **Vec2D**. Hint: The code needs four special methods in total, including the constructor.

Fill in your answer here

1	Solution at the end of the exam set.
---	--------------------------------------

Maximum marks: 8

12 Differenslikninger

We have the following system of difference equations for $n \geq 0$:

$$\begin{aligned}x_{n+1} &= x_n + y_{n+1} \\ y_{n+1} &= y_n + k x_n (x_n - 1)\end{aligned}$$

The systemet is called the *Bogdanov map*, after the Russian mathematician Rifkat Bogdanov.

a) Write a function **bogdanov(N, k, x0, y0)**, which computes the solution (x_n, y_n) to the difference equation for $n = 1, 2, \dots, N$. The arguments to the function are the number of steps N , the parameter k , and the initial values x_0 and y_0 . The function shall return the solution as a tuple (x, y) , where x is a list or array with the solution values $x_0, x_1, x_2, \dots, x_N$ and y is a list or array with solution values $y_0, y_1, y_2, \dots, y_N$.

b) Write code where you first use the function from question a) to solve the system for $N = 1000$, $x_0 = y_0 = 0.1$ and two different values of k ; $k = 1.2$ and $k = 2.0$. Then, write code to plot both solutions in the same window, with the x -values on the x -axis and the y -values on the y -axis.

Fill in your answer here

1	Solution at the end of the exam set.
---	---

Maximum marks: 10

13 Personregister

The file "register.txt" contains information about persons, on the format:

12345, Alice Johnson, 25, alice.johnson@example.com, USA

23456, Bob Smith, 30, bob.smith@example.com, Canada

78912, Robert Grønn, 25, robert.gronn@example.com, Norway

a) Write a function **read_file(filename)** that takes a file name as argument, reads a file on the format given above, and returns a nested dictionary. The keys of the dictionary shall be the id number (first number in each line), and the value shall be a dictionary containing the rest of the information about each person, with keys "name", "age", "email" and "country".

b) Write a function **print_emails(registry)** which takes a dictionary on the format from question a) as argument, and writes a list with all email addresses to a file "emails.txt", with one email address per line.

c) Write a function **filter_country(registry, country)** that takes two arguments: A dictionary on the format described in a), and a text string that contains the name of a country. The function shall return a dictionary with all persons in the register that belong to the given country. If there are no people in the register from the given country, the function shall return an empty dictionary.

Fill in your answer here

1	Solution at the end of the exam set.
---	--------------------------------------

Maximum marks: 9

14 Forlengs Euler metoden

An ordinary differential equation (ODE) can be written on the general form:

$$u'(t) = f(t, u),$$

If we know the initial value $u(0) = u_0$, we can solve such an ODE with the Forward Euler method, given by the following formula:

$$u_{n+1} = u_n + \Delta t f(t_n, u_n)$$

Here $u_n = u(t_n)$ is the estimated solution at time step n , $u_{n+1} = u(t_{n+1})$ is the solution at the next step, and Δt is the time step.

Write a function **forward_euler(f, T, N, u0)** which uses the Forward Euler method to solve an ODE written on the form given above from time $t=0$ to $t=T$, with N time steps and initial value u_0 . The argument f is the function that defines the right hand side of the ODE, implemented as a Python function. The **forward_euler** function shall return two arrays or lists; one containing the time points $t_0, t_1, \dots, t_{N+1}$, and one with the solution $u_0, u_1, \dots, u_{N+1}$.

Fill in your answer here

1	
	Solution at the end of the exam set.

Maximum marks: 5

```

"""
Question 6
"""

from math import factorial,exp

#a)
def exp_approx(x,n):
    s = 0
    for k in range(n+1):
        s += x**k / factorial(k)
    return s

#b)
def test_exp_approx():
    k = 10
    x = 2.0
    tol = 1e-4
    expected = exp(x)
    computed = exp_approx(x,k)
    print(expected, computed)
    assert abs(expected - computed) < tol

```

```

"""
Question 7
"""

from math import log
import numpy as np

N = 100
start = 1
stop = 10

#a)
dx = (stop-start)/(N)
x = []; y = []

for i in range(N+1):
    x.append(start + dx*i)
    y.append(log(x[i]))

print(len(x), len(y))
print(x,y)

#b
x = np.linspace(start, stop, N + 1)
y = np.log(x)
print(len(x), len(y))
print(x,y)

"""
Question 8
"""

```

```
def triangle_area(vertices):
    (x1, y1), (x2, y2), (x3, y3) = vertices
    return 0.5 * abs(x2*y3 - x3*y2 - x1*y3 + x3*y1 + x1*y2 - x2*y1)
```

"""

Question 9

"""

```
import matplotlib.pyplot as plt
```

#a)

```
class Diff:
    def __init__(self, f, h = 1e-4):
        self.f = f
        self.h = h

    def __call__(self, x):
        return (self.f(x + self.h) - self.f(x))/self.h
```

```
def f(x):
    return np.sin(2 * np.pi * x)
```

#b)

```
df = Diff(f)
x = np.linspace(0,1,101)
plt.plot(x,df(x))
plt.show()
```

"""

Question 10

"""

```
class Piecewise:
    def __init__(self, a, b):
        self.a = a
        self.b = b

    def __call__(self, x):
        for i, b_ in enumerate(self.b):
            if x < b_:
                return self.a[i]
        return self.a[-1]
```

```
def test_Piecewise():
    pwf = Piecewise([1, 2, 3], [10,20])

    expected = [1,2]
    computed = [pwf(5),pwf(15)]

    for e, c in zip(expected, computed):
        assert e == c
```

```
"""
```

Question 11

```
"""
```

```
class Vec2D:
    def __init__(self, comps):
        self.x = comps[0]
        self.y = comps[1]

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Vec2D([x,y])

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __mul__(self, other):
        return self.x * other.x + self.y * other.y
```

```
x = Vec2D([1,2])
y = Vec2D([4,4])
```

```
z = x + y
print(z)
#(5,6)
print(type(z))
#<class 'Vec2D'>
print(x * y)
#12
```

```
"""
```

Question 12

```
"""
```

#a)

```
def bogdanov(N, k, x0, y0):
    x = np.zeros(N + 1)
    y = np.zeros(N + 1)

    x[0] = x0
    y[0] = y0

    for n in range(N-1):
        y[n + 1] = y[n] + k * x[n] * (x[n] - 1)
        x[n + 1] = x[n] + y[n + 1]

    return x, y
```

#b)

```
x1, y1 = bogdanov(N = 1000, k = 1.2, x0 = 0.1, y0 = 0.1)
x2, y2 = bogdanov(N = 1000, k = 2.0, x0 = 0.1, y0 = 0.1)
```

```
plt.plot(x1, y1)
```

```
plt.plot(x2, y2)
plt.show()
```

```
"""
```

Question 13

```
"""
```

#a)

```
def read_file(filename):
    keys = ['name', 'age', 'email', 'country']
    register = {}

    with open(filename) as infile:
        for line in infile:
            words = line.split(',')
            id = int(words[0])
            name = words[1].strip()
            age = int(words[2])
            email = words[3].strip()
            country = words[-1].strip()
            info = dict(zip(keys, [name, age, email, country]))
            register[id] = info
    return register
```

#b)

```
def print_emails(register):
    with open('emails.txt', 'w') as outfile:
        for info in register.values():
            outfile.write(info['email'] + '\n')
```

#c)

```
def filter_country(register, country):
    filtered = {}

    for id, info in register.items():
        if info['country'] == country:
            filtered[id] = info
    return filtered
```

```
"""
```

Question 14

```
"""
```

```
"""
```

The question text contains a small error. It says that the function shall return to arrays t and u , both having indices from 0 , to $N+1$. However, since there should be N time steps, both arrays should have length $N+1$, and the indices run from 0 to N . This typo in the question text is taken into account when marking the exam.

```
"""
```

```
def forward_euler(f, T, N, u0):
    t = np.linspace(0, T, N+1)
```

```
u = np.zeros_like(t)
dt = t[1] - t[0]
u[0] = u0
for n in range(0, N):
    u[n+1] = u[n] + dt * f(t[n], u[n])
return t, u
```